

Friday we took memory apart: address-of, dereference, and the fact that `a[i]` is `*(a + i)`. Monday was Labor Day, so that was nine days ago.

Today: the control structures are the easy half. Loops and conditionals look almost exactly like Python with braces. The half that is not familiar is what happens when you hand a variable to a function, which is where the rest of the hour goes, written live rather than shown finished.

PART

1. Conditionals and loops, quickly
2. `switch`, and why it needs `break`
3. Live: a copy, or the variable itself

Part 1. Conditionals and loops, quickly

`if / else if / else`, `while`, `for`, `break`, `continue`. Nothing here is new except the braces and the parentheses.

The one genuine difference: **the condition is an `int`**. Zero is false, anything else is true, and that is why `if (x = 5)` compiles, assigns, and is always true. Write it on the board once so they recognise it later.

Part 2. `switch`, and why it needs `break`

```
switch (grade) {
  case 'A': cout << "excellent"; break;
  case 'B': cout << "good";      break;
  default:  cout << "hmm";
}
```

Talk to your neighbor. What happens if you leave out a `break`?

It falls through into the next case and keeps going. Not a bug in the language, a deliberate feature that catches everyone once.

Part 3. Live: a copy, or the variable itself

Leave the deck here and write this with them.

```
void tryToChange(int n) { n = 99; }  
void reallyChange(int &n) { n = 99; }  
  
int x = 1;  
tryToChange(x);      cout << x << endl;    // 1  
reallyChange(x);     cout << x << endl;    // 99
```

One character of difference and two entirely different behaviours.

Before running it: what does each line print, and why?

Print `&x` inside the caller and `&n` inside each function. Without the `&` the addresses differ, because the function got a copy. With the `&` they are the same address, because the function got the variable.

A slide can assert this. Two addresses on screen prove it. Friday names the three ways to pass and when each one is right.